



MODULE 7

iOS Application Architecture

Enterprise Mobile Application Development Bootcamp | AI-Integrated Edition

Phase 02 | iOS Development with Swift

Module 7 Roadmap

Six blocks. By the end: a refactored, testable, dependency-injected PNC Mobile feature.



MVC vs. MVVM

- The Massive View Controller Problem
- Extracting the View Model
- the “Zero UIKit Imports” Test



The Massive View Controller Problem

```
class AccountsViewController: UIViewController {
    override func viewDidLoad() {
        super.viewDidLoad()

        URLSession.shared.dataTask(with: accountsURL) { data, _, error in
            guard let data else { return }
            let accounts = try? JSONDecoder().decode([Account].self, from: data)
            DispatchQueue.main.async {
                self.accounts = accounts ?? []
                for account in self.accounts where account.balance < 0 {
                    self.showOverdraftWarning(for: account)    // business rule
                }
                self.tableView.reloadData()                    // UI update
            }
        }.resume()
    }
    // ...plus DataSource, Delegate, formatting, error alerts, navigation...
}
```



Model-View-ViewModel (MVVM)

- This is an architecture pattern that separates concerns into separate objects
- The **Model** implements the structure of the data being used in the screen
- The **View** displays the content for the screen
- The **ViewModel** implements all application logic for the screen
- There is no absolute consensus on what is MVVM you can find discussions/arguments over details about what should go where
- The overall goal is to separate data (Model), presentation (View), and logic (ViewModel)



MVVM: Extracting the View Model

```
import Foundation    // <- not UIKit

final class AccountsViewModel {
    private let repository: AccountsRepository
    private(set) var accounts: [Account] = []
    var onAccountsUpdated: (() -> Void)?

    init(repository: AccountsRepository) {
        self.repository = repository
    }
    func loadAccounts() async {
        accounts = (try? await repository.fetchAccounts()) ?? []
        onAccountsUpdated?()
    }
    var overdraftAccounts: [Account] {
        accounts.filter { $0.balance < 0 }
    }
}
```



Block 1 - MVC vs. MVVM

Skill Check ✓

- Name three distinct responsibilities crammed into the Massive View Controller example.
- What's the literal, checkable test for whether the view has leaked into the business logic layer?
- In the MVVM version, what's left for the view controller to actually do?



Clean Architecture & Layering

- The Dependency Rule
- Presentation, Domain & Data Layers
- Mapping Layers onto an iOS Project



The Dependency Rule

Presentation

Views, ViewModels - what the user sees and interacts with

depends on ↓

Domain

Business rules, use cases, models - the core of the app, no framework imports

depends on ↓

Data

Repositories, API clients, persistence - where data actually comes from

Dependencies point downward only. Domain never imports from Presentation or Data.



Mapping Layers onto a Real iOS Project



Presentation/

AccountListView.swift, AccountsViewModel.swift



Domain/

Account.swift, AccountsRepository.swift (protocol only)



Data/

RemoteAccountsRepository.swift, AccountsAPIClient.swift

Notice: the Domain layer defines AccountsRepository as a protocol. The Data layer provides the concrete implementation. This inversion is what makes the dependency point inward - Domain doesn't know Data exists.



Block 2 - Clean Architecture & Layering

Skill Check ✓

- State the Dependency Rule in one sentence.
- Which layer is allowed to import UIKit or SwiftUI, and which layer must never import either?
- Why does the Domain layer define AccountsRepository as a protocol rather than a concrete class?



The Coordinator Pattern

- Navigation Logic Trapped in View Controllers
- Extracting a Coordinator
- Who Owns the Flow



Coordinators in UIKit

- In UIKit, a coordinator has the following responsibilities:
- It controls a view's context
 - i.e. whether it is used inside of a UINavigationController, a UITabBarController or just as is
- It defines which transitions are performed when certain events occur
 - Such as presenting a new view controller or pushing it onto the current navigation stack
- It creates view controllers and manages the memory exchange between them
 - e.g. by creating new view models



The Problem: Navigation Logic Trapped in View Controllers

- This screen now has a hidden dependency on a screen that isn't its own - pure coupling introduced by where the navigation code happened to be written.
- AccountDetailViewController now has to know TransferViewController exists, how to construct it, and what happens when it finishes.

Code Bite

```
class AccountDetailViewController:
    UIViewController {
    @objc func transferButtonTapped() {
        let transferVC =
            TransferViewController(fromAccount: account)
        transferVC.onComplete = { [weak self] in
            self?.navigationController?
                .popToRootViewController(animated: true)
        }
        navigationController?
            .pushViewController(transferVC, animated:
            true)
    }
}
```



The Coordinator: One Object Owns the Flow

- Define a function that will do the navigation in a delegate protocol

Code Bite

```
protocol AccountDetailDelegate: AnyObject {  
    func accountDetailDidRequestTransfer(  
        from account: Account)  
}
```



Coordinators in SwiftUI

- A coordinator in *SwiftUI* consists of **two components**:
 - a coordinator *view*
 - and a coordinator *object*
- In SwiftUI, we have two different responsibilities to be taken care of by a coordinator:
 - A view's context needs to be controlled by a View that is part of the view hierarchy
 - And we need to control a view's lifecycle
- This can be achieved easily with an ObservableObject
- We can easily pass it around by reference and make it conform to the Identifiable protocol.



Implementing The Coordinator

- AccountDetailViewController calls `delegate?.accountDetailDidRequestTransfer(from:)` and knows nothing else about what screen comes next.

Code Bite

```
final class AccountsCoordinator {  
    private let navController: UINavigationController  
  
    func showTransfer(from account: Account) {  
        let vc = TransferViewController(fromAccount:  
account)  
        vc.onComplete = { [weak self] in  
            self?.navController  
                .popToRootViewController(animated: true)  
        }  
        navController.pushViewController(vc, animated:  
true)  
    }  
}
```



Block 3 - The Coordinator Pattern

Skill Check ✓

- What hidden coupling does putting navigation code directly in a view controller create?
- After adopting a Coordinator, what does AccountDetailViewController know about TransferViewController?
- Who decides what screen comes next in a Coordinator-based flow?



Repository Pattern & Service Layers

- Abstracting Data Sources
- Where Business Logic Actually Lives
- ViewModel → Repository → Service → API



The Repository Pattern



Repository protocol

Defines what data is available - no mention of HOW it's fetched



Remote implementation

Fetches from the network via an API client



Local/cached implementation

Reads from disk or Core Data - same protocol, different source



Old School Asynchronicity

- Apps frequently have need to execute tasks that take a while to complete
- You do not want the main thread to be blocked waiting for the task to finish
 - The UI would be unresponsive
- One pattern to handle this is to pass a callback to the function that initiates the long-running task
 - When the task finishes the initiating function can invoke the callback
 - It will pass the results into the callback
 - This allows the calling code to determine what to do with the results
- Closures are often used



Code Bite

```
func processData(cust: Customer) {  
    . . .  
}  
  
repository.getById(id: 125,  
    callback: processData)  
  
repository.getById(id: 125) { c:Customer in  
    . . .  
}
```

async/await

- Functions declared with the `async` keyword can pause (suspend) themselves
- Inside such functions the `await` keyword will pause execution without blocking the main thread
 - Execution will resume when the `async` function that is being awaited completes
 - The function being awaited must itself be declared with `async`
- This approach can greatly reduce nested closures and "callback hell"

Code Bite

```
class Repository {  
    func getById(id: Int) async -> Customer {  
        . . .  
    }  
}  
  
let cust = await repository.getById(125)
```



Service Layers: Where Business Logic Actually Lives

```
struct TransferEligibilityService {  
    func canTransfer(amount: Decimal, from account: Account) -> Result<Void, TransferError> {  
        guard amount > 0 else { return .failure(.invalidAmount) }  
        guard account.balance >= amount else { return .failure(.insufficientFunds) }  
        return .success(())  
    }  
}
```

// ViewModel calls the service, doesn't reimplement the rule itself

```
func attemptTransfer(amount: Decimal) {  
    switch eligibilityService.canTransfer(amount: amount, from: sourceAccount) {  
    case .success: performTransfer(amount: amount)  
    case .failure(let error): presentError(error)  
    }  
}
```



Service Layers

- Business logic is implemented in service classes
 - Code that could conceivably be used in multiple places in the application
 - Code that could conceivably be used in other applications
- Code that is specific to a particular view belongs in a ViewModel not a service object
- Service objects should conform to a protocol
 - This allows for the substitution of different implementations when necessary



Putting It Together: the Full Chain



- The View only knows the ViewModel.
- The ViewModel only knows a Service protocol and a Repository protocol.
- Nothing upstream knows how data is actually fetched - that knowledge stays entirely inside the Repository's concrete implementation.



Block 4 - Repository & Service Layers

Skill Check ✓

- What's the one-sentence difference between what a Repository does and what a Service does?
- In the full chain, what does the View directly depend on?
- Why can the Repository's data source change from network to cache without the ViewModel changing at all?



Dependency Injection

- Why Not Just Use Singletons Everywhere?
- _INITIALIZER Injection
- Protocol-Based DI for Testability



Why Not Just Use Singletons Everywhere?

- A singleton hard-codes which implementation you get. If `AccountsRepository.shared` always talks to the real network, how do you test a `ViewModel`'s overdraft logic without a live server?
- Hard to test - always hits the real network

Code Bite

```
class AccountsViewModel {  
  
    func loadAccounts() async {  
        accounts = try? await  
            AccountsRepository.shared.fetchAccounts()  
    }  
  
}
```



The Answer: Dependency Injection

- The dependency is a protocol, supplied from outside the class
- This allows a fake version to be injected during tests instead of the real version that would be injected at runtime

Code Bite

```
class AccountsViewModel {  
    private let repository: AccountsRepository  
  
    init(repository: AccountsRepository) {  
        self.repository = repository  
    }  
  
    func loadAccounts() async {  
        accounts = try? await  
            repository.fetchAccounts()  
    }  
}
```



Protocol-Based DI for Testability

```
protocol AccountsRepository {  
    func fetchAccounts() async throws -> [Account]  
}
```

```
final class RemoteAccountsRepository: AccountsRepository {  
    let apiClient: AccountsAPIClient  
    func fetchAccounts() async throws -> [Account] {  
        try await apiClient.fetchAccounts()  
    }  
}
```

```
final class FakeAccountsRepository: AccountsRepository {  
    let stubbedAccounts: [Account]  
    func fetchAccounts() async throws -> [Account] { stubbedAccounts }  
}
```



Initializer Injection In Use

```
// Production
```

```
let viewModel = AccountsViewModel(  
    repository: RemoteAccountsRepository(apiClient: AccountsAPIClient())  
)
```

```
// Unit test - no network, no server, fully deterministic
```

```
let viewModel = AccountsViewModel(  
    repository: FakeAccountsRepository(  
        stubbedAccounts: [Account(name: "Test", balance: -50)]  
    )  
)  
  
await viewModel.loadAccounts()  
XCTAssertEqual(viewModel.overdraftAccounts.count, 1)
```



Initializer Injection Alternatives

- The problem with initializer injection is that the code that creates the instance needs to know which protocol implementation should be used
 - This works for a view creating an instance of a ViewModel
 - It does not work for the ViewModel creating an instance of a service class
- A better approach for those situations is to distribute the object through Environment injection
 - This is SwiftUI's native key-value pair injection system
- This approach comes in two forms:
 - Environment injection
 - Environment object injection



Environment Injection - setup

- It is most convenient to add a property to the EnvironmentValues type
 - This property will provide the instance of the type being injected
- The EnvironmentValues type stores data in key/value pairs
- The key must be an EnvironmentKey
 - A struct value is defined before the extension can add the property to EnvironmentValues

Code Bite

```
struct NetworkServiceKey: EnvironmentKey {
    static let defaultValue: NetworkService =
        ProductionNetworkService()
}
extension EnvironmentValues {
    var networkService: NetworkService {
        get { self[NetworkServiceKey.self] }
        set {
            self[NetworkServiceKey.self] =
newValue
        }
    }
}
```



Environment Injection - defining and injecting

- You can use the `#if DEBUG` conditional compilation directive to inject different implementations during development vs. release
- If you don't configure an injected type the default value (from the injection key) will be used

Code Bite

```
// Inject it at the Root View
RootView()
    .environment(\.networkService,
                MockNetworkService())

// Consume it anywhere downstream
struct DeepChildView: View {
    @Environment(\.networkService)
    var service

    var body: some View { ... }
}
```



Block 5 - Dependency Injection

Skill Check ✓

- What specifically makes a singleton-based dependency hard to unit test?
- What's the minimum change needed to make AccountsViewModel testable without a real network call?
- What does the ViewModel depend on: a concrete class, or a protocol? Why does that distinction matter?



Offline-First Design & State Management Strategies

- Cache-Then-Network
- Sync Queues
- Revisiting Single Source of Truth at Application Scale



Offline-First Design Concepts



Cache-then-network

Show cached data immediately, then silently refresh from the network and update if it changed



Sync queue

Actions taken offline (e.g. a transfer) are queued and replayed once connectivity returns



Optimistic UI

Show the result of an action immediately, reconcile with the server response after



State Management Strategies: Revisited at Application Scale

**A Repository isn't just about testability.
It's the application-wide single source of truth
that Module 6 introduced at the screen level.**

- This is the thread that ties the whole module together:
 - MVVM separates concerns
 - DI makes the pieces swappable and testable
 - The Repository gives the app one source of truth
 - Offline-first design is what that architecture makes achievable.



Block 6 - Offline-First & State Strategy

Skill Check ✓

- Why is 'show an error and stop' an inadequate offline strategy for a core banking feature?
- What's the difference between cache-then-network and optimistic UI?
- How does the Repository pattern from Block 5 directly enable offline-first design?



Refactor: TransferViewController

Skill Check ✓

- Starter file contains a Massive View Controller mixing networking, validation, and UI.
- Extract a TransferViewModel with zero UIKit imports.
- Extract transfer eligibility rules into a TransferEligibilityService.
- Inject an AccountsRepository protocol via the ViewModel's initializer - no singletons.
- The refactored ViewModel must be unit-testable using a fake repository, with no real network call.



Module 7 Complete

You can now recognize a Massive View Controller and extract an MVVM structure from it, reason about Clean Architecture's dependency rule, remove navigation logic into a Coordinator, inject dependencies for testability instead of reaching for singletons, split data access and business rules into Repository and Service layers, and design for offline-first use. Module 8 builds the real networking and persistence layer this architecture was designed to hold.

Next: Module 8 - Data & API Integration